

COMPLETE PROJECT GUIDE & TECHNICAL REPORT

Meridian Commission Analytics Suite

An end-to-end FP&A sales commission calculation, data-prep pipeline, and analytics system — built to demonstrate the complete technical scope of the Sr. Financial Analyst, FP&A (Sales Compensation) role at Applied Systems, Inc.

Prepared by: Bharath Chandra

Target role: Sr. Financial Analyst — Financial Planning & Analysis, Applied Systems, Inc.

Project type: Independent portfolio project (synthetic data, fictional company)

Built with: Excel, Python, JavaScript/HTML, Git/GitHub — assisted by Claude (Anthropic)

Table of Contents

1	Executive Summary	3
2	The Role This Project Targets	4
3	The Business Problem	5
4	Project Architecture — What Was Built	6
5	Tools & Technologies Used	7
6	The Dataset — How It Was Built	8
7	FP&A Concepts & Techniques Applied	9
8	Calculations & Formulas — Full Technical Walkthrough	10
	8.1 The Marginal Tiered Commission Formula	
	8.2 Aggregation Formulas — SUMIFS, COUNTIFS, AVERAGEIFS	
	8.3 Lookup Formulas — INDEX/MATCH	
	8.4 Forecasting — TREND / Linear Regression	
	8.5 Net Revenue Retention (NRR)	
	8.6 Budget Variance Formulas	
	8.7 Scenario / What-If Formulas	
	8.8 Exception / Outlier Detection Logic	
9	Component Deep Dive — The Excel Workbook (10 tabs)	13
10	Component Deep Dive — The Python ETL Pipeline	14
11	Component Deep Dive — The Interactive Dashboard	15
12	Component Deep Dive — The PDF Collateral	16
13	Data Integrity — The Three-Way Cross-Check Methodology	17
14	Alternative Tools & Platforms — How Else This Could Be Built	18
15	Detailed Mapping to the Applied Systems Job Description	19
16	Key Findings & Business Insights	20
17	Honest Scope — What This Project Is and Isn't	21
18	How to Access, Run, and Deploy This Project	22
19	Closing Summary	23

1 Executive Summary

A plain-language summary of what this project is, why it exists, and what it proves — for a reader who knows nothing about it yet.

This document describes an independent portfolio project called the **Meridian Commission Analytics Suite**. It was built by Bharath Chandra to demonstrate, concretely and verifiably, the technical skills listed in the job description for a **Sr. Financial Analyst, Financial Planning & Analysis (FP&A) — Sales Compensation** role at **Applied Systems, Inc.**, a Chicago-based insurance technology company.

Rather than list skills on a resume, this project *builds the actual work product* a person in that role would produce: a commission calculation engine, a data-cleansing pipeline, forecasting and budget-variance models, a scenario-planning tool, an automated exception/audit review, an interactive analytics dashboard, and Sales-facing communication documents. Every number in every one of these deliverables is calculated from a single shared dataset, and the math was independently rebuilt in three different tools — Excel, Python, and JavaScript — to prove the calculations are correct and internally consistent, the same way a real FP&A team would reconcile a number before it reaches a VP or a board deck.

Because there is no real company involved, the entire dataset is **synthetic** — invented for this project, seeded so it reproduces identically every time it's regenerated, and built around a fictional company called **Meridian Risk Technologies**, a B2B SaaS business selling cloud policy-management software to insurance agencies (the same market Applied Systems itself serves). Fifteen fictional sales reps, 270 fictional closed deals across the first half of a fictional 2026 fiscal year, and a documented commission plan make up the raw material every other piece of this project is built from.

What exists at the end of this project

Deliverable	Format	What it demonstrates
Commission & FP&A Model	Excel workbook, 10 tabs	Commission calculation, forecasting, scenario planning, audit controls, budgeting, executive reporting
Data Prep Pipeline	Python script	Data cleansing, blending, transformation — the "Alteryx" skill listed as a plus in the JD
Interactive Dashboard	Single HTML file, live on the web	Self-service analytics with filters, drill-downs, and live scenario modeling — the "Tableau" skill listed as a plus
Commission Plan Quick Reference	One-page PDF	Sales-facing plan communication — the JD's "educate Sales on commission programs" requirement
Rep Commission Statements	15-page PDF, one page per rep	Individual payout transparency — the JD's "sales rep visibility" requirement
This report	PDF	Full documentation of the above, for any reader

The single most important fact about this project: the total FY26 H1 commission expense — **\$916,935**, on **\$14,074,500** of bookings ARR — was calculated three separate, independent times (once in Excel formulas, once in a Python pipeline, once in JavaScript) and all three produced the exact same number, down to the cent. That agreement is not a coincidence or a copy-paste; it is the result of implementing the same business logic correctly in three different languages and cross-checking the outputs against each other, which is precisely the kind of reconciliation discipline the target role is asking for.

2 The Role This Project Targets

Every design decision in this project traces back to one job posting. Here it is, in full, for context.

Position: Sr. Financial Analyst | **Team:** Financial Planning and Analysis | **Company:** Applied Systems, Inc. | **Location:** Hybrid — Chicago, Illinois (one in-office day per week) | **Salary range:** \$81,000–\$110,000 USD

What the job posting says the role will do

- Lead company-wide commission calculations, ensuring accuracy, timeliness, and alignment with business objectives.
- Drive commission analytics and financial modeling to identify trends, optimize performance, and support strategic decision-making.
- Enhance commission processes, logic, and reporting to improve efficiency, accuracy, and sales rep visibility.
- Partner cross-functionally to implement compensation plans, educate Sales on commission programs, and support forecasting and budgeting through commission accruals.

What the job posting says the ideal candidate looks like

- 5+ years of FP&A and business partnership experience, preferably in software, internet, SaaS, or related industries.
- Bachelor's degree in Finance, Business, or a related field; MBA and/or CPA preferred.
- Advanced financial analysis, modeling, and forecasting skills, including expertise in Excel and complex model development.
- Strong analytical, problem-solving, and business acumen, with a curious, proactive approach to identifying opportunities, mitigating risks, and tackling new challenges.

Bonus points called out explicitly

- Exceptional problem-solving and analytical skills, particularly an understanding of key financial and operational "needle movers."
- SaaS business industry experience and familiarity with the metrics behind it.
- **Familiarity with Alteryx, Tableau, and AI tools — specifically Claude.**

Why this matters for how the project was designed: almost every technical bullet above maps to a specific, buildable artifact — a calculation engine (commission accuracy), a modeling layer (analytics/trends), a communication layer (rep visibility/educating Sales), a forecasting layer (accruals/budgeting), and two named tools (Alteryx, Tableau) that have functional equivalents which don't require a paid license to demonstrate. The project was scoped explicitly against this list, bullet by bullet, rather than being a generic finance portfolio piece retrofitted to look relevant.

Company context, for completeness

Applied Systems is a 40+ year old company building cloud and AI-powered software for insurance agencies and brokers. The posting emphasizes a people-first culture, welcomes candidates from non-traditional backgrounds (including those without a degree or military spouses), and notes that AI tools are already used internally in parts of the recruiting workflow — which is part of why building this project with an AI tool (Claude) is a relevant, honest data point to raise in conversation about this role, not an incidental detail.

3 The Business Problem

Every FP&A project should start with a problem, not a tool. Here is the (fictional but realistic) problem this project solves.

Meridian Risk Technologies is a growing B2B SaaS company that sells cloud-based policy management software to insurance agencies and brokers. Its sales organization has grown to 15 reps spread across three segments — SMB, Mid-Market, and Enterprise — each with its own quota and territory. Like most companies at this stage, its commission process grew up informally: a single spreadsheet, maintained by hand, where a finance analyst looks up each rep's bookings, manually checks which commission tier they've hit, types in the payout, and repeats this for every rep, every month.

That manual process has four specific, recurring problems — the same four problems that show up at almost any company before FP&A puts a real system in place:

Problem	Why it happens	What it costs the business
Slow month-end close	Every rep's attainment and tier has to be looked up and calculated by hand	Multiple days of analyst time each month spent on lookups instead of analysis
Inconsistent results	Manual tier lookups are error-prone, especially near tier boundaries	Reps get overpaid or underpaid; trust in the comp plan erodes
No forward visibility	The spreadsheet only shows what already happened, not what's coming	Finance can't answer "what will next quarter's commission expense be?" for budgeting
No self-service for Sales	Reps have to ask a person every time they want to know their numbers	Manager and analyst time spent answering the same questions repeatedly

This project addresses all four directly: an automated calculation engine replaces the manual lookups; the same engine, built three ways, removes the risk of calculation error; a forecasting and scenario layer gives finance forward visibility; and a dashboard plus individual statements give reps and managers self-service access to their own numbers.

Why this maps so directly to the Applied Systems role: the JD's very first bullet is "lead company-wide commission calculations, ensuring accuracy, timeliness, and alignment with business objectives" — which is, almost word for word, a description of fixing exactly this problem. The rest of the JD's bullets (analytics, process improvement, forecasting, educating Sales) are the natural next steps once the core calculation problem is solved, which is why the project builds all of them, not just the calculation engine in isolation.

4 Project Architecture — What Was Built

A map of every piece, how data flows between them, and why each one exists.

The project has one foundational dataset and four layers built on top of it. Every layer reads from the same source data, which is the entire point: rather than four disconnected demos, this is one coherent system where a change at the source would ripple consistently through everything downstream.

Layer 0 — The Dataset (the foundation)

Three CSV files: a sales roster (15 reps, their quotas, segments, regions), a set of closed deals (270 records across January-June 2026), and the commission plan rules (the tier rates). A fourth file, a hypothetical annual budget, was added later to support budget-variance analysis. Every other layer reads from these files, or from data derived from them.

Layer 1 — The Data Prep Pipeline (Python)

Takes the raw deal data — which deliberately contains realistic messiness (a sign error, a missing value, a duplicate record, inconsistent text casing) — and cleans it, blends it with the roster, calculates commission, and writes out clean, ready-to-use tables. This is the "Alteryx-equivalent" layer: a structured, auditable, step-by-step data workflow.

Layer 2 — The Excel Model (the core FP&A deliverable)

A 10-tab workbook that independently recalculates commission from the same raw data using live formulas, then builds seven more tabs on top of that calculation: a roster and plan-rules reference, a management summary, a monthly forecast, a scenario planner, an audit/exceptions view, a budget-vs-actual view, and an executive dashboard with charts.

Layer 3 — The Interactive Dashboard (the "Tableau-equivalent")

A single, self-contained HTML file that loads the same dataset into a browser and lets anyone explore it live: filter by quarter or segment, sort the rep table, drag sliders to model a different commission plan, and see the same exception flags the Excel workbook produces — all without needing Excel installed or a Tableau license.

Layer 4 — Sales-Facing Communication (PDFs)

Two documents built for an audience outside finance: a one-page plain-language guide explaining the commission plan to reps, and an individual one-page commission statement for each of the 15 reps, generated automatically from the same underlying numbers.

How data flows between the layers

Raw CSVs → Python Pipeline (cleans, blends, calculates) → clean output tables, **and separately** → Excel Bookings Detail tab (recalculates independently with formulas) → feeds every other Excel tab, **and separately** → Dashboard JSON payload (same raw data, recalculated a third time in JavaScript) → feeds every dashboard tab, **and** → PDF generators (read the Python pipeline's clean output) → the two PDFs.

Note the phrase "recalculates independently" appearing three times. This is not an accident of the build process — it's the central design decision of the whole project, covered in full in Section 13.

5 Tools & Technologies Used

Every tool used in this project, what role it played, and why it was the right choice for that role.

CORE FP&A DELIVERABLE

Microsoft Excel (built via openpyxl)

The commission model, forecast, scenario planner, audit view, and budget comparison all live in a single .xlsx workbook. Excel remains the lingua franca of FP&A — it's what gets emailed to a VP, reviewed in a meeting, and audited by Sales. The workbook itself was *authored programmatically* using `openpyxl`, a Python library that writes real .xlsx files with live formulas, formatting, conditional formatting, and native charts — not a static export. Opening the file in Excel shows actual formulas in every cell, fully auditable and editable, exactly as if a person had built it by hand.

DATA PREPARATION — THE "ALTERYX" EQUIVALENT

Python 3, with pandas and NumPy

`pandas` is the standard Python library for tabular data manipulation — filtering, grouping, joining, and aggregating data, the same operations Alteryx's visual tools perform, just expressed as code instead of drag-and-drop nodes. `NumPy` was used specifically to generate the synthetic dataset — its random-number generation functions (with a fixed seed, so results are reproducible) created realistic performance variance across reps, deal-size distributions, and the intentional data-quality issues.

INTERACTIVE ANALYTICS — THE "TABLEAU" EQUIVALENT

HTML5, CSS3, vanilla JavaScript, and Plotly.js

The dashboard is a single HTML file — no server, no build process, no installation. It embeds the dataset directly as JSON inside the page, and vanilla JavaScript (no framework like React or Vue) filters, aggregates, and re-renders the view live in the browser. **Plotly.js**, loaded from a public CDN, renders the actual charts — bar charts, line charts, pie charts — the same category of visual Tableau would produce, but reachable by anyone with a browser and no license.

SALES-FACING DOCUMENTS

HTML/CSS rendered to PDF via wkhtmltopdf

The commission plan one-pager and the 15 rep statements were authored as styled HTML (same visual design system as the dashboard, for brand consistency) and converted to PDF using `wkhtmltopdf`, a command-line tool that renders HTML/CSS through a WebKit browser engine and outputs a print-accurate PDF. This let the PDFs share exact colors, fonts, and table styling with the dashboard, rather than being built in a separate tool with a different look.

VERSION CONTROL & HOSTING

Git and GitHub (with GitHub Pages)

The entire project is a Git repository with a real commit history, hosted publicly on GitHub. **GitHub Pages** serves the dashboard's HTML file directly as a live, clickable website — <https://bharath097.github.io/meridian-commission-analytics/> — so the project isn't just files on a laptop; it's a deployed, shareable product.

BUILD & VERIFICATION ENVIRONMENT

Node.js (for testing), LibreOffice (for Excel recalculation), Python (pdf2image, cairosvg)

Node.js was used specifically to run the dashboard's real JavaScript logic outside the browser, feeding it the actual dataset and checking its outputs against known values — a genuine automated test, not a visual guess. LibreOffice's headless mode was used to force a full formula recalculation of the Excel file and catch any formula errors before delivery. `pdf2image` and `cairosvg` converted PDFs and diagrams to images so every visual output in this project could be inspected before being called finished.

AI ASSISTANCE

Claude (Anthropic)

The entire project — data generation, formula design, Python code, JavaScript, HTML/CSS, the diagrams, the PDFs, and this report — was built in collaboration with Claude. This is a direct, honest answer to the JD's "familiarity with AI tools, specifically Claude" bonus point, and a genuine demonstration of how to use an AI tool for real analytical and technical output rather than just chat.

Why this specific stack, and not something else

Every tool above was chosen because it produces a *real, usable artifact* a hiring manager or a Sales rep could open without installing anything unusual, or because it directly stands in for a named tool in the job posting. Section 14 covers, in detail, how this same project could instead be built with Alteryx, real Tableau, Power BI, SQL, Anaplan, or several other platforms — and the trade-offs of each.

6 The Dataset — How It Was Built

Where every number in this project ultimately comes from.

Nothing in this project uses real company data — there is no real "Meridian Risk Technologies." The dataset was generated by a Python script (`generate_data.py`) using a fixed random seed, meaning it produces the exact same numbers every single time it's run. This matters for two reasons: it makes the project fully reproducible by anyone who runs the script, and it means every calculation in this report can be checked against a stable, known set of numbers.

The sales roster

15 reps were generated across three segments: **6 SMB reps** (smaller deals, lower quota), **6 Mid-Market reps** (mid-sized deals, mid quota), and **3 Enterprise reps** (large deals, high quota). Each rep was assigned a realistic quarterly quota with built-in variance ($\pm 12\%$) to reflect that territories are never perfectly identical, a hire date (affecting how large their existing "book of business" is, since a longer-tenured rep has more renewal-eligible accounts), a region, and a starting book of business used later for renewal and retention calculations.

The deals

272 deals were generated across January–June 2026 (270 after data cleaning removed 2 bad records — see below), split into three types:

- **New Business** — a brand-new customer signing on. This is what counts against a rep's quota.
- **Upsell / Expansion** — an existing customer buying more. Sized as a percentage of the rep's existing book.
- **Renewal** — an existing contract renewing, with realistic variance (some renew flat, some shrink, some grow) built in via a weighted random Net Revenue Retention factor per deal.

Each rep was also given a random "performance factor" (drawn from a normal distribution) so that some reps realistically overperform and some underperform their quota — this is what produces the wide, realistic spread of attainment outcomes (from 17% to 178% across the 30 rep-quarters) that the exception-detection logic in Section 8.8 is built to catch.

The deliberately broken data

Six specific data-quality issues were injected on purpose, because a data-cleansing pipeline with nothing to clean isn't demonstrating anything:

Issue injected	Real-world equivalent	How the pipeline handles it
One negative ARR value	A sign error from a CRM export or manual entry mistake	Detected and corrected (converted to absolute value)
One trailing-whitespace customer name	Inconsistent data entry	Trimmed
One lowercase rep ID	Case-inconsistent system exports	Standardized to uppercase
One missing (null) ARR value	An incomplete record	Dropped, since guessing a dollar figure risks over/underpaying a rep
One exact duplicate record	The same deal exported twice	Detected and removed
One inconsistent-case, padded deal type ("renewal ")	Free-text entry inconsistency	Standardized via trim + title-case

The commission plan rules

A five-row table defining the tiered New Business structure (8% / 10% / 12%) and the flat Upsell (6%) and Renewal (3%) rates — described in full technical detail in Section 8.1.

The budget (added for the Budget vs. Actual feature)

A hypothetical month-by-month FY26 budget was created separately, deliberately *not* matching the actuals exactly — because a budget that perfectly predicts reality isn't realistic. Some months beat budget by a wide margin (February, +43%), others missed (June, −12%), which is what makes the variance analysis in Section 8.6 meaningful rather than trivial.

7 FP&A Concepts & Techniques Applied

The actual financial-analysis techniques this project demonstrates — not just the tools used to build them.

1. Marginal (tiered) commission structuring

A commission plan where the rate increases as a rep hits higher levels of quota attainment, and — critically — each dollar is paid at the rate for the bracket it falls into, not a single blended rate applied to the whole amount. This is standard practice in SaaS sales compensation because it rewards overperformance without overpaying underperformance, and it's the single most technically involved calculation in the whole project (full walkthrough in Section 8.1).

2. Quota attainment analysis

Tracking what percentage of quota each rep hit, by quarter, and using that both to drive commission (via the tiers above) and to drive management action — flagging reps who are dramatically over or under their target for a coaching or territory-design conversation.

3. Net Revenue Retention (NRR)

A core SaaS metric measuring how much revenue from an existing customer base was retained (or lost) over a period, independent of new customer acquisition. An NRR above 100% means existing customers are growing faster than they're churning; below 100% means the existing base is shrinking even before counting new sales. Full formula in Section 8.5.

4. Bookings mix analysis

Breaking total ARR booked into New Business, Upsell, and Renewal, because these three have very different strategic implications — New Business shows market penetration, Upsell shows account expansion, Renewal shows retention — and a healthy business needs all three in the right proportion, not just a big total number.

5. Commission-to-revenue ratio (a cost-of-sale metric)

Tracking commission expense as a percentage of the ARR it produced (here, roughly 6.5%) is how finance monitors whether the cost of the sales organization is staying within the assumptions built into the company's overall financial plan — a rate that drifts upward without an explained reason is an early warning sign worth investigating.

6. Driver-based forecasting

Rather than guessing next quarter's commission expense, this project forecasts the driver (bookings ARR, via a linear trend on six months of actuals) and then applies the trailing commission rate to that forecasted driver — a standard FP&A forecasting pattern, because forecasting the underlying business activity and then applying a rate is more defensible than forecasting an expense number directly.

7. Budget-to-actual variance analysis

Comparing what was planned (budget, set before the year began) against what actually happened, month by month, and explaining the gap — the single most common recurring deliverable in real FP&A work, and the reason Section 8.6 and the Budget vs. Actual tab exist.

8. Scenario / sensitivity analysis ("what-if" modeling)

Testing how a change to an input assumption (a commission rate, a quota level) would have changed the outcome, holding everything else constant — used in real FP&A work to evaluate a proposed plan change *before* proposing it to leadership, rather than finding out the cost impact after the fact.

9. Exception-based review (a financial control)

Rather than manually reviewing all 270 individual deals or all 30 rep-quarters, defining clear rules for what counts as unusual (attainment far outside the normal range, a deal size far outside a rep's typical pattern) and having the system surface only those — the standard way real finance teams manage review effort at scale without missing the outliers that matter.

10. Data governance and reconciliation

Cleaning messy source data before it enters a financial calculation, and then independently rebuilding the same calculation in a second (and third) system to catch errors before a number is presented — covered in full in Section 13, and arguably the most important technique demonstrated in the entire project.

8 Calculations & Formulas — Full Technical Walkthrough

Every significant formula used anywhere in this project, exactly as written, with the business reason it exists — not just what it does, but why that's the correct way to do it.

8.1 The Marginal Tiered Commission Formula

This is the single most important calculation in the entire project — everything else is built around getting this right. The commission plan pays New Business at three rates depending on how much of quarterly quota a rep has hit: 8% on the first 100% of quota, 10% on the portion between 100% and 125%, and 12% on anything above 125%. The critical design detail is that this is **marginal**, not a single blended rate — a rep who reaches 110% of quota does not earn 10% on their entire New Business ARR; they earn 8% on the first 100%-worth and 10% only on the extra 10%-worth. Paying a single blended rate the moment a rep crosses a threshold would create a "cliff" that distorts a rep's behavior near quarter-end (an incentive to delay or rush deals to land on the right side of a threshold) — marginal tiering avoids that.

The formula is built from three pieces, computed for each rep, each quarter:

```
Tier 1 amount = MIN(New Biz ARR, Quota)
```

```
Tier 2 amount = MAX(MIN(New Biz ARR, Quota × 1.25) - Quota, 0)
```

```
Tier 3 amount = MAX(New Biz ARR - Quota × 1.25, 0)
```

```
Commission = Tier1 × 8% + Tier2 × 10% + Tier3 × 12%
```

Why each piece is shaped this way: the `MIN()` in Tier 1 caps that bracket at exactly one quota-worth of ARR, no matter how much the rep actually closed — so a rep at 200% attainment still only gets the 8% rate on the first 100%-worth, not on everything. The Tier 2 formula subtracts the Quota from a capped amount (`MIN(New Biz ARR, Quota×1.25)`) so it isolates just the slice between 100% and 125% — and the outer `MAX(..., 0)` prevents this from going negative for a rep who hasn't even reached 100% yet (their Tier 2 amount correctly becomes zero, not a negative number that would corrupt the total). Tier 3 mirrors that same logic for everything above 125%.

Where this formula actually lives: exactly this logic was written three separate times — as Excel formulas (`MIN` / `MAX` nested in the Bookings Detail and Scenario Planner tabs), as a Python function (`marginal_new_biz_commission()` in the ETL pipeline), and as a JavaScript function (inside `scenarioTotalCommission()` and the deal-level formula in the dashboard). All three independently produce the same answer.

Worked numeric example

A rep with a \$150,000 quarterly quota closes \$180,000 in New Business (120% attainment):

Bracket	Formula result	Rate	Commission
Tier 1 (0-100%)	$\text{MIN}(180,000, 150,000) = 150,000$	8%	\$12,000
Tier 2 (100-125%)	$\text{MAX}(\text{MIN}(180,000, 187,500) - 150,000, 0) = 30,000$	10%	\$3,000
Tier 3 (125%+)	$\text{MAX}(180,000 - 187,500, 0) = 0$	12%	\$0
Total			\$15,000

Upsell and Renewal commission are far simpler by design — they use a single flat rate (`ARR × 6%` and `ARR × 3%` respectively) rather than tiers, because expansion and retention revenue is treated as valuable at any volume, without needing an accelerator to encourage more of it the way New Business does.

8.2 Aggregation Formulas — SUMIFS, COUNTIFS, AVERAGEIFS

Nearly every summary number in this project — a monthly total, a rep's quarterly bookings, an average deal size — is produced by conditionally summing or counting rows in the 270-row deal table that match certain criteria. Excel's `SUMIFS`, `COUNTIFS`, and `AVERAGEIFS` functions do exactly this: sum/count/average a range, but only for rows where one or more other columns match specified conditions.

```
=SUMIFS(Bookings!$K$5:$K$274, Bookings!$B$5:$B$274, "R001", Bookings!$G$5:$G$274, "Q1", Bookings!$H$5:$H$274, "New Business")
```

This single formula answers: "of all 270 deals, sum the ARR column, but only for rows where the rep is R001, the quarter is Q1, and the deal type is New Business." It's how the Rep-Quarter Summary tab, the Monthly Accrual tab, and the dashboard's month/type breakdowns are all built — the same technique, applied with different filter criteria each time.

`AVERAGEIFS` is used specifically in the deal-level outlier detection (Section 8.8): it computes a rep's *typical* deal size for a given deal type, which then becomes the baseline a specific deal's size is compared against to flag it as unusually large.

Why this approach instead of, say, a manual filter-and-copy process: these formulas are *live* — if a single deal in the source data changes, every summary number that depends on it recalculates automatically, with no risk of someone forgetting to refresh a filtered view or a copy-pasted subtotal going stale.

8.3 Lookup Formulas — INDEX/MATCH

Whenever one tab needs to pull a single piece of information about a rep — their name, segment, or quota — from the roster tab, based only on knowing their Rep ID, this project uses `INDEX/MATCH` rather than the more commonly-taught `VLOOKUP`:

```
=INDEX('Sales Roster & Quotas'!$C$6:$C$20, MATCH($B5, 'Sales Roster & Quotas'!$B$6:$B$20, 0))
```

`MATCH` finds the row position of a given Rep ID within the roster's ID column; `INDEX` then returns whatever value sits in that same row position of a different column (here, the rep's name). **Why INDEX/MATCH over VLOOKUP:** `VLOOKUP` requires the lookup column to be the leftmost column in its range and breaks if a column gets inserted or reordered; `INDEX/MATCH` has no such restriction, making the workbook more resilient to future editing — a meaningful consideration in a workbook meant to be maintained over time, not used once.

8.4 Forecasting — TREND / Linear Regression

The Q3 forecast (July–September) is built using Excel's `TREND` function, which fits a straight line through the six months of actual bookings data and projects that line forward:

```
=TREND($B$5:$B$10, ROW($B$5:$B$10)-ROW($B$5)+1, 7)
```

The middle argument re-indexes the six actual months as x-values 1 through 6, so the final argument (7, 8, 9) requests the forecasted y-value at month 7, 8, and 9 along that same line. Mathematically, `TREND` is performing ordinary least-squares linear regression — the same underlying math implemented by hand in both the Python pipeline (using basic algebra on means and sums) and the JavaScript dashboard (an explicit slope/intercept calculation), so all three forecasts agree.

The forecasted commission expense is then `Forecasted Bookings × Trailing 6-Month Average Commission Rate` — a driver-based approach (Section 7, technique #6) rather than forecasting the commission dollar figure directly.

8.5 Net Revenue Retention (NRR)

NRR measures what happened to the revenue that already existed at the start of a period, independent of new sales. It's calculated here as:

$$\text{NRR} = \text{Renewed ARR} \div \text{Eligible-to-Renew ARR}$$

where **Eligible-to-Renew ARR** is approximated as half of a rep's starting book of business (since, on average, a book renews in roughly quarterly cycles over the year, two quarters of data means roughly half the book was up for renewal). An NRR above 100% for a segment (as this project finds for SMB, at ~125%) means expansion and upgrade activity within the existing base outpaced any churn; below 100% (as found for Enterprise, at ~97%) means the segment is losing slightly more revenue from existing accounts than it's gaining from them.

Why this matters enough to calculate carefully: NRR is one of the single most-watched metrics in SaaS finance, because it answers a question New Business bookings alone cannot: is the revenue the company already has getting healthier or eroding? A company can post strong new bookings while its underlying retention quietly deteriorates — NRR is the metric that catches that.

8.6 Budget Variance Formulas

Comparing actual (or forecasted) performance against a budget set before the year began uses simple, direct formulas — the value of this technique is in the discipline of doing it consistently every month, not in formula complexity:

$$\text{Variance \$} = \text{Actual} - \text{Budget} \quad \text{Variance \%} = \text{Variance \$} \div \text{Budget}$$

The one piece of care taken here is **honesty about what's being compared**: for January–June, "Actual" means real closed-deal data; for July–September, since those months haven't happened yet in the fictional timeline, the comparison is explicitly labeled against the *forecast*, not a real actual — a "Basis" column on both the Excel tab and the dashboard tab makes this distinction visible on every row, so no one could mistake a projected number for a confirmed one.

8.7 Scenario / What-If Formulas

The Scenario Planner recomputes total commission expense under a hypothetical plan (different tier rates, or an adjusted quota) while holding actual booking behavior fixed. For each of the 30 rep-quarters, the same marginal-tier formula from Section 8.1 is re-run, but with the tier rates and quota swapped out for whatever the user has set on the sliders or input cells:

$$\begin{aligned} \text{Scenario Quota} &= \text{Base Quota} \times (1 + \text{Quota Adjustment \%}) \\ \text{Scenario Tier1/2/3} &= \text{same MIN/MAX logic as 8.1, using Scenario Quota} \\ \text{Scenario Commission} &= \Sigma (\text{Tier1} \times \text{Rate1} + \text{Tier2} \times \text{Rate2} + \text{Tier3} \times \text{Rate3} + \text{Upsell} \times \text{UpsellRate} + \text{Renewal} \times \text{RenewalRate}) \text{ across all 30 rep-quarters} \end{aligned}$$

Why bookings are held constant: this is a deliberate and important modeling choice. The scenario answers "what would we have paid, given the deals that actually closed, under a different plan?" — which is the correct question to ask when evaluating a proposed plan change against historical performance. It does *not* attempt to predict how reps would have behaved differently under a different incentive structure (a much harder, more speculative question), and the documentation is explicit about that boundary rather than overclaiming.

8.8 Exception / Outlier Detection Logic

Two independent flagging rules, each built for a different kind of risk:

Rep-quarter attainment flags — three plain conditions, evaluated in priority order:

```
IF(Attainment > 150%, "Accelerator review",  
  IF(Attainment < 60%, "Coaching flag",  
    IF(Renewal ARR = 0, "Retention gap", "")))
```

150% and 60% were chosen as thresholds because they represent genuine outliers relative to the ~97% average attainment this dataset produces — not arbitrary round numbers. A rep at 150%+ either has an unusually easy territory (worth revisiting the quota) or is a genuine top performer worth recognizing; a rep under 60% for a full quarter is a clear coaching signal rather than normal variance.

Deal-level size outliers — comparing each deal's ARR to that specific rep's own average deal size for that specific deal type:

$$\begin{aligned} \text{Ratio} &= \text{Deal ARR} \div \text{AVERAGEIFS(all deals by this rep, of this deal type)} \\ \text{Flag if Ratio} &\geq 2.5 \end{aligned}$$

Why compare against the rep's own average, not a company-wide average: a \$500,000 Enterprise deal is completely normal for an Enterprise rep and would be a wild anomaly for an SMB rep — comparing every deal to one company-wide number would either miss real Enterprise outliers or falsely flag every normal Enterprise deal. Comparing each deal only to that same rep's own typical pattern makes the flag meaningful regardless of segment.

Both exception rules were implemented independently in Excel and in the dashboard's JavaScript. Run against the same 270-deal dataset, they flag the exact same three rep-quarters and the exact same nine deals in both places — a real, tested cross-check, not an assumption.

9 Component Deep Dive — The Excel Workbook

All 10 tabs, what each one does, and why it exists.

#	Tab	Purpose
1	Cover	Purpose statement, methodology notes, tab guide, and a color-coding legend (blue = input, black = formula, green = cross-sheet link) so anyone opening the file understands its conventions immediately.
2	Commission Plan Rules	The tiered rate table as named, editable, highlighted assumption cells — changing a rate here recalculates the entire workbook, because every downstream formula references these cells rather than hardcoding numbers.
3	Sales Roster & Quotas	The 15-rep roster: segment, region, quota, starting book of business — the reference table every lookup formula in the workbook points back to.
4	Bookings Detail	The calculation engine itself. All 270 deals, one row each, with the full marginal-tier formula (Section 8.1) computed live, per row. This is the tab every other tab ultimately sources its numbers from.
5	Rep-Quarter Summary	A management-review rollup: 30 rows (15 reps × 2 quarters), each summarizing that rep-quarter's bookings, attainment, and commission — with conditional-formatting color scales making outliers visually obvious at a glance.
6	Monthly Accrual & Forecast	Monthly bookings and commission for Jan-Jun (actual), plus a TREND-based Jul-Sep forecast, for cash and expense accrual planning.
7	Executive Dashboard	KPI cards and four native Excel charts (monthly trend, attainment by rep, ARR mix, NRR by segment) — the one-page view built for a leadership audience, not an analyst working session.
8	Scenario Planner	What-if modeling on plan rates and quota (Section 8.7), with a live base-vs-scenario comparison chart.
9	Audit & Exceptions	The two exception-flagging rules from Section 8.8, applied to all 30 rep-quarters and all 270 deals, with Excel Tables and filters so a reviewer can isolate just the flagged rows.
10	Budget vs Actual	The hypothetical FY26 budget compared month-by-month to actuals/forecast (Section 8.6), with an honestly-labeled basis column and a grouped bar chart.

Formula discipline across the whole workbook

Every number that could be a formula is a formula — there are no hardcoded totals or copy-pasted values anywhere downstream of the raw input cells (which are themselves clearly marked in blue). This was verified, not just intended: the workbook was run through a full recalculation check after every change, and it currently contains **6,355 live formulas across the 10 tabs with zero calculation errors**.

Visual and usability conventions

- **Color coding:** blue text = an input someone can safely change; black = a calculated formula; green = a value pulled from another tab.
- **Conditional formatting:** color scales on attainment columns, data bars on commission columns, and highlight fills on flagged exception rows — so a reviewer's eye is drawn to what matters without reading every cell.
- **Print setup:** every tab is formatted for landscape printing with fit-to-width scaling, because a real workbook eventually gets printed or exported to PDF for a meeting, and a workbook that only looks right on-screen isn't finished.

10 Component Deep Dive — The Python ETL Pipeline

The data-prep layer, structured deliberately like a visual workflow tool.

The pipeline (`pipeline.py`) is written as six discrete, named steps, each with a single responsibility — the same structure a tool like Alteryx enforces visually, tool-by-tool on a canvas, expressed here as functions:

Step	What it does	Alteryx-equivalent
1. Input Data	Loads the three source CSVs (deals, roster, plan rules)	Input Data tool
2. Data Cleanse	Trims whitespace, standardizes text case, fixes sign errors, drops nulls, removes duplicates	Data Cleansing + Filter tools
3. Join	Blends the cleaned deals with the roster on Rep ID, so every deal carries its rep's segment, quota, and region	Join tool
4. Formula	The marginal-tier commission calculation (Section 8.1), applied per deal	Formula / Multi-Row Formula tools
5. Summarize	Builds four rollups: by rep-quarter, by month, by deal type, and NRR by segment	Summarize tool
6. Output Data	Writes five clean CSVs plus a run log documenting exactly what was cleaned	Output Data tool

Each step logs what it actually did — not just that it ran, but the specific counts (e.g. `sign_errors_corrected: 1`, `duplicates_removed: 1`) — which is what makes the pipeline auditable rather than a black box. Running it produces this exact console output:

```
[CLEANSE] sign_errors_corrected: 1
[CLEANSE] missing_arr_dropped: 1
[CLEANSE] duplicates_removed: 1
[CLEANSE] Record count 272 -> 270
[FORMULA] Commission calculated for 270 deals (total: $916,935)
```

Why Python instead of a GUI tool: the logic is identical to what a visual ETL tool would express, but written as code, it's version-controlled (every change to the pipeline logic has a Git history), runs without any paid license, and can be executed identically on any machine with Python installed — a meaningful practical advantage when the goal is a project someone else can actually run themselves, not just a screenshot of a workflow canvas.

11 Component Deep Dive — The Interactive Dashboard

Seven tabs, all reading the same dataset, all recalculating live in the browser.

Tab	What it shows
Overview	KPI strip (bookings, commission, commission %, attainment), monthly trend chart, ARR mix pie chart, NRR-by-segment chart — filterable live by Quarter and Segment
Rep Performance	A horizontal attainment chart sorted low-to-high, and a sortable rep detail table (click any column header to re-sort)
Deal Trends	Deal volume by month, ARR by product line, average deal size by type
Forecast	The same TREND-based Q3 forecast as the Excel model, shown as a combined actual+forecast chart with a written method note
Budget vs Actual	The budget comparison from Section 8.6, as a live grouped bar chart plus a detail table
Scenario Planner	Six live sliders (three New Biz tiers, Upsell rate, Renewal rate, quota adjustment) that instantly recompute and re-chart the base-vs-scenario commission comparison as they're dragged
Exceptions	The same two flagging rules as the Excel Audit tab, shown as live counts and filtered tables (only flagged rows are displayed, not all 270/30 rows)

How it actually works, technically

The entire dashboard is **one HTML file**. The dataset (all 270 deals, the roster, and the budget) is embedded directly inside the page as a JSON block — meaning the file is fully self-contained and works by simply opening it in a browser, with no server, database, or installation required. JavaScript functions filter, group, and aggregate this embedded data on the fly, in the browser, every time a filter changes or a slider moves — recomputing from the raw deal-level data each time, not from a pre-baked static summary.

A deliberate resilience fix

During testing, a scenario was deliberately simulated where the charting library (Plotly, loaded from an external CDN) fails to load — e.g. a corporate firewall blocking it. In the original design, that single failure would have thrown an error that silently stopped every subsequent part of the page from rendering, including tables that don't even need a charting library. This was caught, and fixed: each of the dashboard's render functions now runs independently, wrapped so that one failure is logged and skipped rather than cascading — meaning even if charts can't load, the Exceptions table, the Scenario comparison numbers, and every other non-chart element still work correctly. This fix was tested by deliberately removing the charting library in a simulated environment and confirming the rest of the page still produced correct numbers.

12 Component Deep Dive — The PDF Collateral

The two documents built for an audience outside finance — Sales reps themselves.

Commission Plan Quick Reference (one page)

A plain-language explanation of the tiered commission structure, including the exact same worked numeric example from Section 8.1, written for a Sales rep with no finance background — not for an analyst. It also answers the three questions reps most commonly ask (when they're paid, whether renewals count toward quota, and how a deal spanning two quarters is treated) and states clearly who to contact for quota questions versus calculation disputes. This directly answers the JD's requirement to "educate Sales on commission programs."

Rep Commission Statements (15 pages, one per rep)

An individual, automatically-generated one-page statement for every rep — their H1 bookings and commission broken out by quarter and deal type, and a complete log of every deal they closed with its individual commission amount. Generated directly from the same clean data the Python pipeline produces, so every number on every statement traces back to the exact same source as every other deliverable in the project. This is the concrete artifact behind the JD's "sales rep visibility" requirement — a rep opening their page doesn't have to trust a summary number, they can see exactly which deal produced exactly how much commission.

Both PDFs were built as styled HTML (sharing the same navy/gold/slate visual identity as the dashboard) and converted with `wkhtmltopdf`. The rep with the most deals in H1 (25 closed deals) was specifically checked to confirm their statement still fit cleanly on one page before this was considered finished — an edge case, not just the average case.

13 Data Integrity — The Three-Way Cross-Check Methodology

The single design decision that makes this project's numbers trustworthy, explained in full.

The exact same commission calculation — the marginal-tier formula from Section 8.1 — was implemented **three separate times, by hand, in three different languages**: once as native Excel formulas, once as a Python function inside the ETL pipeline, and once as a JavaScript function inside the dashboard. Each implementation was written independently against the same business rule description, not copied from one to the other. Then all three were run against the identical 270-deal dataset and their outputs compared.

Build method	Total H1 Commission	Total H1 Bookings ARR
Excel formulas (Bookings Detail tab)	\$916,935	\$14,074,500
Python pipeline (pandas)	\$916,935	\$14,074,500
JavaScript (dashboard, tested via Node.js)	\$916,935 (cent-level rounding: \$916,934.99)	\$14,074,500

This same cross-check discipline was then extended to the newer features: the Scenario Planner's default output was confirmed to exactly reproduce the base plan total in both Excel and the dashboard, and a test scenario (+10% quota adjustment) was confirmed to produce the identical adjusted total (\$911,250) in both places. The Exceptions logic was confirmed to flag the exact same three rep-quarters and the exact same nine deals in both Excel and the dashboard.

Why this matters more than any individual formula

A single formula, however cleverly written, could still contain a logic error that nobody catches. The value of building the same calculation multiple independent ways is that a mistake made in one implementation is extremely unlikely to be made identically in a second, unrelated implementation — so agreement between them is meaningful evidence of correctness, not just confidence. This is the same principle behind why real accounting uses double-entry bookkeeping, why real software engineering uses independent test suites, and why a real FP&A team reconciles a number two ways before it goes in front of a VP or a board.

Verification wasn't limited to the numbers matching each other

Beyond cross-checking the three implementations against each other, each individual deliverable was separately verified on its own terms:

- The Excel workbook was run through a full formula recalculation after every change, with zero tolerated errors (6,355 formulas, 0 errors, confirmed via LibreOffice's headless recalculation engine).
- The dashboard's JavaScript was executed outside the browser, in Node.js, against the real embedded dataset, and its outputs checked against known values — an actual automated test, not a visual guess.
- A deliberate failure scenario (a blocked external library) was simulated to confirm the dashboard degrades gracefully instead of breaking entirely.
- The Python pipeline and Excel workbook were both re-run from a completely clean, freshly extracted copy of the project — first in this sandbox, and then a second time by the project's owner on their own separate Windows machine — with both runs producing the identical \$916,935 figure.

14 Alternative Tools & Platforms — How Else This Could Be Built

No tool choice is the only correct one. Here is how this same project could be built differently, and the real trade-offs of each path.

Layer	Used in this project	Alternative(s)	Trade-off
Commission calc & FP&A model	Excel (openpyxl)	Google Sheets + Apps Script	Google Sheets is free and natively collaborative, but its formula engine and charting are less powerful than Excel's, and it's less universally expected as the FP&A deliverable format in enterprise settings.
		Anaplan / Adaptive Insights (Workday)	Purpose-built EPM/planning platforms with native multi-dimensional modeling and workflow — the real tools many large companies use for commission and planning. Far more powerful for enterprise scale, but require an expensive license and administrative setup, making them impractical for an independent portfolio project.
		VBA/macro-driven Excel	Could replicate the same logic using VBA procedural code instead of native formulas. Less transparent and auditable than live formulas (a reviewer has to read code, not just click a cell), so formulas were preferred here.
Data prep / ETL	Python (pandas)	Real Alteryx Designer	The actual named tool in the JD. A visual canvas is faster to build for non-programmers and easier for a less technical stakeholder to follow step-by-step. Requires a paid license; the Python version reproduces the same workflow structure and logic without one, and is easier to version-control.
		SQL (dbt, Snowflake, BigQuery)	The standard approach at companies with a real data warehouse — cleansing and transformation expressed as SQL/dbt models. More scalable for large data volumes and integrates with a company's existing data stack, but is overkill for a 270-row dataset and requires warehouse infrastructure this project doesn't have.
Interactive dashboard	HTML/JS + Plotly.js	Real Tableau / Tableau Public	The actual named tool in the JD, with more polished built-in interactivity and a familiar interface for business users. Requires a license (or Tableau Public's data-must-be-public limitation) and produces a file format that needs Tableau installed to edit — the HTML version runs anywhere with just a browser and is fully version-controlled as plain text.
		Power BI	Strong choice in Microsoft-centric environments and integrates tightly with Excel. Similar licensing consideration to Tableau, and less universally viewable than a plain web page.
		Looker / Looker Studio	Google's BI tool, strong for semantic-layer modeling at scale. Free tier exists (Looker Studio) but is heavier-weight than needed here and ties the dashboard to a Google account.
Sales-facing PDFs	HTML/CSS → wkhtmltopdf	Microsoft Word mail-merge, or a real HR/comp platform (Xactly, CaptivateIQ, Varicent)	Dedicated commission platforms like CaptivateIQ or Xactly are what companies actually use in production to calculate and distribute statements at scale, with built-in approval workflows and rep self-service portals. They require a paid subscription and integration work; the HTML/PDF approach here demonstrates the same end output (a rep-facing statement) without that infrastructure.
Hosting / delivery	GitHub + GitHub Pages	Netlify, Vercel, a company intranet	Any static-hosting platform would work equally well for the dashboard specifically; GitHub was chosen because it also provides free, public version control and a familiar code-review-style history in the same place.

The honest reason for this project's specific tool choices: every alternative above that was *not* used shares one thing in common — it requires a paid license, dedicated infrastructure, or an account the reader of this report doesn't have. The tools actually used were chosen so that anyone — a recruiter, a hiring manager, a future teammate — could open, run, and verify every single piece of this project themselves, for free, using only things already on their computer or reachable through a browser.

15 Detailed Mapping to the Applied Systems Job Description

Every line of the job posting, matched to the specific piece of this project that addresses it.

"What You'll Do"

JD requirement	Where it's addressed in this project
Lead company-wide commission calculations, ensuring accuracy, timeliness, and alignment with business objectives	The Bookings Detail tab's deal-level calculation engine (Section 9), independently cross-checked three ways for accuracy (Section 13)
Drive commission analytics and financial modeling to identify trends, optimize performance, and support strategic decision-making	The Executive Dashboard, the interactive dashboard's Overview/Rep Performance/Deal Trends tabs, and the Scenario Planner (Sections 9, 11)
Enhance commission processes, logic, and reporting to improve efficiency, accuracy, and sales rep visibility	The Commission Plan Rules tab's editable-assumption architecture, the Audit & Exceptions tab, and the individual Rep Commission Statements (Sections 9, 12)
Partner cross-functionally to implement compensation plans, educate Sales on commission programs, and support forecasting and budgeting through commission accruals	The Commission Plan Quick Reference PDF (educating Sales, Section 12) and the Monthly Accrual & Forecast / Budget vs Actual tabs (forecasting and budgeting, Sections 8.4, 8.6)

"What We're Looking For"

JD requirement	Where it's addressed
Advanced financial analysis, modeling, and forecasting skills, including expertise in Excel and complex model development	The full 10-tab, 6,355-formula Excel workbook with zero recalculation errors (Section 9)
Strong analytical, problem-solving, and business acumen, with a curious, proactive approach to identifying opportunities, mitigating risks, and tackling new challenges	The Audit & Exceptions logic (Section 8.8), which proactively surfaces risks rather than waiting to be asked, and the specific, named findings in Section 16
5+ years of FP&A and business partnership experience	<i>Not addressed — see Section 17</i>
Bachelor's degree in Finance, Business, or related field; MBA/CPA preferred	<i>Not addressed — see Section 17</i>

"Bonus Points"

JD requirement	Where it's addressed
Understanding of key financial and operational "needle movers"	The commission-to-bookings ratio tracking (Section 7, technique #5) and the specific business interpretation given to each finding in Section 16, not just the raw numbers
SaaS business industry experience and familiarity with the metrics behind it	NRR (Section 8.5), ARR bookings mix, and quota attainment — all tracked and interpreted throughout
Familiarity with Alteryx	The Python ETL pipeline, deliberately structured as an Alteryx-style Input→Cleanse→Join→Formula→Summarize→Output workflow, with an accompanying visual workflow diagram (Section 10)
Familiarity with Tableau	The interactive HTML/JavaScript/Plotly dashboard, live on the web, with the same filtering, drill-down, and self-service analytics pattern (Section 11)
Familiarity with AI tools, specifically Claude	This entire project, including this report, was built using Claude — a direct and literal answer to this exact bullet

16 Key Findings & Business Insights

What the analysis actually found — the difference between building a calculator and doing analysis.

These findings are drawn from the synthetic dataset, but each one represents the kind of observation a real FP&A analyst would be expected to surface and act on, not just report:

One rep is running at 178% of quota. Omar Fischer (Mid-Market) closed Q2 at 177.6% attainment. That could mean he's simply an outstanding performer — but it could also mean his territory's quota was set too conservatively when the plan was built. Both explanations point to the same next step: this is worth a specific conversation with his manager before assuming either explanation is correct.

Two reps have been under 60% attainment for two consecutive quarters. David Nguyen (16.8% in Q2) and Grace Adeyemi (59.7% in Q2) both flagged. Two straight quarters is what distinguishes a genuine coaching need from ordinary quarter-to-quarter variance — a single bad quarter happens to almost everyone; a pattern is what warrants intervention.

SMB segment NRR of 125% is not what it looks like at first glance. A 125% NRR sounds like broad-based strength across the whole segment. Looking at the underlying deals shows it's actually driven by a small number of unusually large expansion renewals, not a broad pattern across all SMB accounts. Reporting the headline number without this context would risk a leadership team drawing the wrong conclusion about the health of the SMB segment as a whole.

Commission expense has held remarkably steady, between 6.1% and 7.2% of bookings ARR every single month of H1. This consistency is precisely what allowed the Q3 forecast to confidently apply a trailing average rate rather than guessing — and it's itself a useful finding: the commission plan is behaving as designed, without unexpected cost creep.

Nine individual deals are 2.5x or larger than that rep's typical deal size for their type. Each one is a legitimate, worth-a-second-look candidate before a commission payout is finalized — not because anything is necessarily wrong, but because an unusually large single deal is exactly the kind of individual transaction a commission audit process exists to catch before payment, not after.

17 Honest Scope — What This Project Is and Isn't

Precision here matters more than anywhere else in this report.

What this project genuinely demonstrates

Every technical and functional requirement in the JD's "What You'll Do" section, and every technical bonus point listed, has a real, working, independently-verified artifact behind it in this project — not a claim, a description of one. A reader can open the Excel file, change a commission rate, and watch the entire workbook recalculate. A reader can open the live dashboard link and drag the scenario sliders themselves. That is a meaningfully different, stronger form of evidence than a bullet point on a resume.

What this project does not, and cannot, demonstrate

Two specific JD requirements are explicitly **not** addressed by this project, and no project — however well built — could address them, because they aren't technical skills:

- **5+ years of FP&A and business partnership experience.** This is a track record requirement, not a skill. A project demonstrates that someone *can* do the work; it cannot demonstrate years actually spent doing it in a real organization, with real stakeholders, real deadlines, and real consequences for getting it wrong.
- **A Bachelor's degree in Finance/Business, MBA, or CPA.** A credential requirement, similarly outside what a project can substitute for.

There's a subtler point worth being just as direct about: the JD's requirement to "partner cross-functionally" and "implement compensation plans" describes an ongoing, interactive, multi-stakeholder *relationship* — working with real Sales leaders, real Legal or HR review, real negotiation over a plan change. A solo project can produce the artifacts that partnership would use (the Quick Reference PDF, the Scenario Planner) but it cannot simulate the actual back-and-forth of doing that work with other people. That distinction is presented honestly, here and in every other document produced for this project (the case study, the recruiter email), rather than implied away.

The intent of this project was never to claim it replaces experience or credentials — it was to remove all doubt about the *technical* half of the hiring decision, so that a conversation about the role can focus on judgment, fit, and experience, rather than whether the underlying Excel and analytical skills are real.

18 How to Access, Run, and Deploy This Project

Everything needed for any reader to verify this project themselves.

Live, no installation required

- **Interactive dashboard:** <https://bharath097.github.io/meridian-commission-analytics/>
- **Full source code and documentation:** <https://github.com/bharath097/meridian-commission-analytics>

Opening the deliverables locally (no setup)

The Excel workbook, the dashboard's HTML file, and both PDFs all open directly — double-click and go. No account, license, or installation is required for any of them.

Running the Python pipeline (one-time setup)

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 pipeline.py
```

This reproduces the identical \$916,935 commission total from a completely clean environment — verified in this project's own build process, and separately re-verified by the project owner on an independent Windows machine, with both runs matching exactly.

Regenerating the dataset from scratch

```
python3 generate_data.py
python3 pipeline.py
```

The random seed is fixed, so this produces the exact same dataset and the exact same \$916,935 result every time — full reproducibility, not a one-off result that can't be checked.

19 Closing Summary

The whole project, in one paragraph.

The Meridian Commission Analytics Suite is a complete, working, independently-verified FP&A system built entirely to demonstrate the technical scope of the Sr. Financial Analyst role at Applied Systems: a commission calculation engine built and cross-checked three separate ways, a data-cleansing pipeline standing in for Alteryx, an interactive live dashboard standing in for Tableau, forecasting and budget-variance models, a scenario-planning tool, an automated audit and exception-review layer, and Sales-facing communication documents — all built from one shared dataset, all publicly hosted, and all reproducible by anyone from a fresh install with a single command. It demonstrates, concretely rather than by assertion, the technical half of what this role asks for, while being explicit and honest about the parts — years of experience, a formal credential, and real cross-functional relationships — that only time in an actual seat can build.

Prepared by Bharath Chandra for consideration for the Sr. Financial Analyst, FP&A position at Applied Systems, Inc. Live dashboard and full source code available at the links in Section 18.